

TECHNICAL DOCUMENTATION

SpeakTwin

AI Communication Mirror
Complete Architecture & Engineering Guide

Version 1.1.0

Stack FastAPI · Whisper · PyTorch · Vanilla JS

Test suite 158 passing

Author Rohan Untawale

Contents

- 1** What SpeakTwin Is
Problem, approach, design constraints
- 2** System Architecture
Layers, boundaries, why capture is client-side
- 3** Request Lifecycle
Every step from microphone to feedback
- 4** Audio Ingestion
Decode, mono, float32, resample
- 5** Acoustic Analysis
Energy, pitch, pause detection — the maths
- 6** Speech-to-Text
Hybrid cloud/local routing
- 7** Linguistic Analysis
Fillers, keywords, clarity
- 8** Scoring & Feedback
Confidence maths and coaching rules
- 9** Session Layer
Aggregation, smoothing, adaptive gating
- 10** Deep Learning Layer
Eight models, measured costs
- 11** Security & Middleware
Rate limits, auth, request IDs
- 12** API Reference
Every endpoint and field
- 13** Configuration Reference
All environment variables
- 14** Datasets & Training
14 corpora, licences, pipeline
- 15** Frontend
Web Audio capture and rendering
- 16** Testing & Deployment
Suite, Docker, CI
- 17** Technology Choices
What is used, and what could be
- 18** Limitations & Roadmap
Honest constraints, next steps
- A** File Map
Every file and its job

1 · What SpeakTwin Is

SpeakTwin is a real-time speech coaching system. It listens while you speak, analyses delivery every 2.5 seconds, and returns actionable feedback — pace, loudness, vocal variety, filler words, clarity, and a composite confidence score.

1.1 The problem

People are poor judges of their own delivery. You cannot hear your own filler words while producing them, you cannot tell that you have drifted monotone, and you rarely notice you have accelerated past the point of intelligibility. Recording yourself helps, but reviewing recordings is slow and most people never do it.

SpeakTwin closes that loop to a few seconds. You speak, and within one chunk you see what a listener would notice.

1.2 Design constraints

Four constraints shape every decision in this codebase:

Constraint	Consequence
Sub-3-second latency	Analysis runs on 2.5 s chunks. Every component has a hard time budget; anything slower than the chunk rate causes backlog.
Must run without a GPU	Default path is pure DSP plus a quantised int8 Whisper. Neural models are optional and off by default.
Must run on a server	A deployed server has no microphone, so capture happens in the browser and audio is POSTed as chunks.
Must degrade, never break	Every optional component — cloud STT, LLM coaching, all eight neural models — fails to a working fallback rather than an error.

1.3 What it measures

6	5	4	8
ACOUSTIC METRICS	LINGUISTIC METRICS	SCORE COMPONENTS	OPTIONAL MODELS

Metric	Unit	Meaning
Energy	RMS + dBFS	Loudness. Reported on a log scale because linear RMS swings wildly with microphone gain.
Pitch	Hz	Fundamental frequency, averaged over voiced frames only.
Pitch variation	Hz (σ)	Vocal expressiveness. Low σ reads as monotone.
Voiced ratio	0–1	Fraction of frames carrying pitch. Distinguishes "monotone" from "barely spoke".
Pause ratio	0–1	Fraction of the chunk that is silence.
Longest pause	seconds	Longest single silent stretch — a hesitation, not just total quiet.
WPM	words/min	Speaking rate from transcript word count over chunk duration.
Filler count / rate	count, per-word	Context-aware filler detection (see §7.1).
Keywords	count	Target vocabulary hits, for positive reinforcement.
Clarity	0–100	MATTR lexical variety minus a filler penalty.
Confidence	0–100	Weighted composite of the four sub-scores.

2 · System Architecture

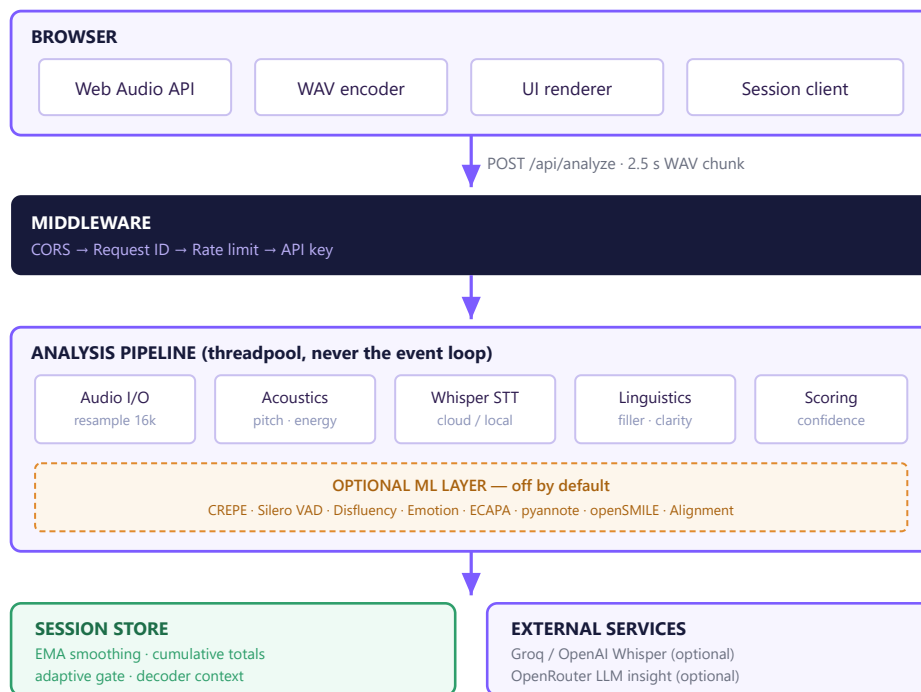


Figure 1 — System architecture. Dashed box is optional and disabled by default.

2.1 Why capture is client-side

The original prototype used `sounddevice` to read the server's microphone. That works on a laptop and is impossible on a deployed server, which has no audio hardware. Capture therefore lives in the browser: the Web Audio API reads the microphone, a hand-written encoder produces WAV, and chunks are POSTed to the backend.

CONSEQUENCE

The backend is stateless with respect to audio hardware and scales horizontally. The legacy server-side capture module is retained for single-machine kiosk use, with its `sounddevice` import guarded so the module stays importable without the package.

2.2 Concurrency model

FastAPI runs `async def` handlers on the event loop. Whisper inference, autocorrelation, and outbound HTTP are all blocking, so running them inline stalls every other request in the process. The handler is `async` only to stream the upload; all real work is dispatched via `run_in_threadpool`.

```
async def analyze_blob(audio_file: UploadFile = File(...), ...):
    raw = await _read_limited(audio_file, settings.max_upload_bytes)
    audio, src_sr = await run_in_threadpool(prepare, raw, settings.max_audio_seconds)
    return await run_in_threadpool(_perform_full_analysis, audio, session_id, src_sr)
```

Models that are not thread-safe — Whisper, and every torch module — are additionally serialised behind per-model locks.

3 · Request Lifecycle

What happens between speaking a word and seeing feedback, in order.

Step 1 — Capture (browser)

On mic activation the frontend requests a 16 kHz `AudioContext`, attaches a `ScriptProcessorNode`, and accumulates samples. When the buffer reaches one chunk (2.5 s × the *actual* sample rate, which may differ from the 16 kHz hint), it is snapshotted and encoded.

DETAIL THAT MATTERS

The browser may ignore the sample-rate hint. The frontend therefore sends `audioCtx.sampleRate` — whatever it truly got — and the backend resamples. Assuming 16 kHz here is precisely the bug described in §4.2.

Step 2 — Session association

On start, the client calls `POST /api/session` and attaches the returned `session_id` to every chunk. Without one, analysis still works but loses cross-chunk smoothing, cumulative totals, decoder context, and the adaptive silence gate.

Step 3 — Upload & validation

The body is streamed in 64 KB reads and aborted the moment it exceeds `MAX_UPLOAD_BYTES`, so an oversized payload is rejected part-way rather than buffered whole. Decoding is capped independently at `MAX_AUDIO_SECONDS`.

Step 4 — Decode & normalise

The header is inspected through `SoundFile` before any samples are read, so only the permitted number of frames is decoded. Audio is mono-mixed, converted to float32, resampled to exactly 16 kHz, and scrubbed of NaN/Inf.

Step 5 — Acoustic analysis

Energy (RMS → dBFS), pitch (windowed autocorrelation with voiced gating and octave correction), and pauses (frame RMS against a dBFS threshold). If the neural layer is enabled, CREPE and Silero override these results here.

Step 6 — Silence gate

If the chunk is silence, transcription is skipped entirely. This matters because Whisper hallucinates confidently on silence. The gate is adaptive per session (§9.3), and when Silero VAD is enabled its verdict supersedes the energy threshold.

Step 7 — Transcription

The resolved engine transcribes the chunk, receiving the tail of the session's previous transcript as `initial_prompt` so words are not mangled at chunk seams. On failure the response is flagged `degraded` and acoustic metrics — which remain valid — are still returned.

Step 8 — Linguistic analysis

Fillers (context-aware), keywords (longest-match n-grams), clarity (MATTR), and WPM from word count over chunk duration.

Step 9 — Scoring & coaching

Four sub-scores are computed and combined by weight. Threshold rules generate coaching messages. If configured and not rate-limited, an LLM produces one punchy headline line; otherwise the first rule-based warning is promoted.

Step 10 — Session fold

The result is folded into the session: totals accumulate, EMAs update, the loudness history extends, and the transcript segment is appended. The updated summary rides back with the response.

Step 11 — Render

The client animates meters, the confidence ring (preferring the smoothed score), the breakdown bars, and appends the transcript with keyword and filler highlighting. The transcript is HTML-escaped before highlight spans are injected.

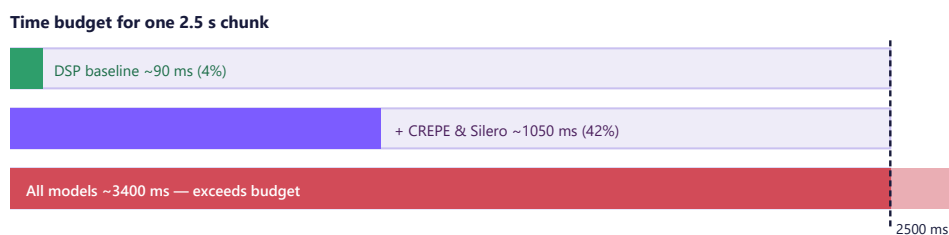


Figure 2 — Measured CPU latency against the fixed chunk budget.

4 · Audio Ingestion

backend/services/audio_io.py

4.1 The contract

Everything downstream assumes mono, float32, exactly 16 kHz. This module is the single place that guarantees it.

```
def prepare(data: bytes, max_seconds, target_sr=16000) -> tuple[np.ndarray, int]:
    mono, src_sr = decode(data, max_seconds=max_seconds) # header-limited read
    audio = resample(mono, src_sr, target_sr)             # polyphase
    audio = np.nan_to_num(audio, nan=0.0, posinf=0.0, neginf=0.0)
    return np.ascontiguousarray(audio, dtype=np.float32), src_sr
```

4.2 The sample-rate bug this module exists to fix

BUG FOUND AND FIXED

The original route read the sample rate and discarded it:

```
y, sr = sf.read(io.BytesIO(audio_bytes)) # sr never used again
```

Everything downstream hardcoded 16 kHz — WPM as `len(audio)/SAMPLE_RATE`, pitch as `sr=SAMPLE_RATE`, and the Groq re-encode. A client sending 44.1 or 48 kHz audio had its pitch and speaking rate wrong by roughly 3×. It only appeared to work because the frontend happened to request 16 kHz.

4.3 Resampling

Polyphase resampling via the reduced integer ratio, so common conversions are exact rather than approximated:

```
divisor = math.gcd(src_sr, dst_sr)
resample_poly(audio, dst_sr // divisor, src_sr // divisor)
# 48000 → 16000 reduces to up=1, down=3 — exact
```

4.4 Bounded decoding

Reading through `SoundFile` rather than `sf.read` allows inspecting the header and decoding only the permitted frames. A one-hour file uploaded to a 30-second endpoint costs 30 seconds of decode, not an hour.

5 · Acoustic Analysis

backend/services/audio_analysis.py — numpy and scipy only, no librosa.

5.1 Energy

RMS amplitude, converted to dBFS for all threshold comparisons:

```
rms = sqrt(mean(x2))
dBFS = 20 * log10(rms)
```

WHY DBFS RATHER THAN LINEAR RMS

Thresholds on linear RMS swing wildly with microphone gain, OS input level, and browser AGC. On a log scale the same thresholds degrade gracefully. The linear constants are *derived* from the dBFS ones, so the two can never drift apart.

Threshold	dBFS	Equivalent RMS	Meaning
SILENCE	−45	≈ 0.0056	No speech detected
LOW	−36	≈ 0.0158	Speaking too softly
HIGH	−12	≈ 0.2512	Speaking too loudly
HARD FLOOR	−60	≈ 0.001	Adaptive gate never goes below this

5.2 Pitch — windowed autocorrelation

64 ms Hann-windowed frames, 32 ms hop. For each frame:

1. **Voiced gate** — frames below −50 dBFS are skipped. Silence has no pitch, and including it corrupts the standard deviation that drives 25% of the confidence score.
2. **Mean removal and windowing** — eliminates DC bias and edge discontinuity.
3. **FFT autocorrelation**, normalised by lag-zero energy.
4. **Restricted lag search** — only lags corresponding to 60–400 Hz are considered.
5. **Voicing threshold** — normalised peak below 0.35 means unvoiced; frame discarded.
6. **Local-maximum check** — see the bug box below.
7. **Octave correction** — autocorrelation peaks at every multiple of the true period, so the raw argmax often lands an octave low. Sub-multiple lags are preferred when at least 85% as strong.
8. **Parabolic interpolation** — sub-sample peak position for finer frequency resolution.

BUG CAUGHT BY THE TEST SUITE

A 20 Hz signal — desk rumble, handling noise, DC drift — was reported as **400 Hz**. Its true period lies outside the search range, so the in-range maximum pins to the boundary lag, and the boundary was taken at face value. A desk bump would trigger "Pitch high, relax your voice". Fixed by requiring the peak to be a genuine interior local maximum.

Robust averaging

The reported mean discards residual octave outliers before averaging — values outside `[median/1.6, median×1.6]` are dropped — so a handful of bad frames cannot drag the reported pitch around.

5.3 Pause detection

Per-frame RMS is computed with a strided view rather than a Python loop, then compared against the dBFS gate. Both the total silent fraction and the longest consecutive run are reported; the latter distinguishes a hesitation from generally sparse speech.

6 · Speech-to-Text

backend/services/speech_to_text.py

6.1 Engine routing

STT_ENGINE	Behaviour
<code>auto</code> (default)	Groq if keyed → OpenAI if keyed → local faster-whisper
<code>groq</code>	Groq <code>whisper-large-v3</code> , falling back to local on failure
<code>openai</code>	OpenAI <code>whisper-1</code> , falling back to local on failure
<code>local</code>	faster-whisper <code>tiny.en</code> , int8 quantised, CPU

6.2 Concurrency

NOT THREAD-SAFE

`WhisperModel.transcribe` is not safe to call from several threads at once, and this service runs inside FastAPI's threadpool. The original code locked only *loading*. Inference is now serialised behind a separate `_inference_lock`.

6.3 Cross-chunk context

Each chunk is decoded with the tail of the session's transcript (220 characters) as `initial_prompt`. Without it, every 2.5 s boundary cuts a word in half and the decoder has no way to recover it. `condition_on_previous_text=False` is set separately to prevent the decoder looping on its own output — the classic repeated-phrase hallucination.

6.4 Silence handling

Two independent defences: the silence gate skips transcription entirely for quiet chunks, and Silero VAD (via `vad_filter`) trims silence before decoding, with a no-VAD retry if the ONNX runtime is unavailable.

7 · Linguistic Analysis

7.1 Context-aware filler detection

`backend/services/filler_detection.py`

Filler words split into two classes. Hesitation sounds are always fillers. Discourse markers have legitimate lexical uses and are only counted in filler positions.

Word	Rule	Counted	Not counted
um , uh , er	always	Hesitation sounds have no lexical use	
so	clause-initial	" So the solution is..."	"... so we shipped it"
well	clause-initial	" Well , that went badly"	"It went well today"
right	clause-final	"That is exactly right "	"The right answer"
like	lexical-context	"it was like really fast"	"I like this", "would like to"

Detection walks the token stream longest-match-first, so `you know` beats a bare `know`, and a recognised phrase used lexically is skipped without re-testing its shorter prefixes.

7.2 Keyword detection

The same longest-match walk over target vocabulary. `artificial intelligence` registers once as a phrase rather than twice as separate words. N-grams never span punctuation, so a phrase cannot straddle a clause boundary.

7.3 Clarity — why MATTR

`backend/services/clarity_analysis.py`

THE MEASUREMENT PROBLEM

Plain type-token ratio (unique ÷ total) is strongly length-dependent. A five-word chunk almost always scores 1.0; a hundred-word one rarely clears 0.6. A per-chunk score and a whole-session score are therefore not comparable on the same axis.

MATTR — moving-average type-token ratio — averages TTR across every sliding 25-word window, which stays stable as text grows. Texts shorter than one window fall back to plain TTR. Raw TTR is still reported for display.

```
mattr = mean(TTR(window) for window in sliding(words, 25))
score = min(1.0, mattr / 0.75) × 100 - filler_rate × 300
```

An intermediate design using root TTR was rejected during development: it under-rewards the short chunks SpeakTwin actually sends, scoring a perfectly clear six-word utterance at 38/100.

8 · Scoring & Feedback

8.1 Confidence composite

backend/services/confidence_score.py

```
score = 100 × ( 0.25·wpm + 0.25·pitch_variation + 0.20·energy + 0.30·fluency )
```

Sub-score	Weight	Curve
WPM	25%	1.0 inside 120–160; falls linearly to 0 at 60 WPM outside the band
Pitch variation	25%	0 at $\sigma=0$; 0.4 at $\sigma=15$; 1.0 at $\sigma\geq 35$, linear between
Energy	20%	Piecewise: 0 below -45 dBFS, ramps to 1.0 at -36 , flat to -12 , decays to 0.4
Fluency	30%	1.0 at zero fillers; 0 at twice the "high" rate (0.16 per word)

DESIGN NOTE

The energy sub-score was a three-step ladder (0.3 / 1.0 / 0.5). A speaker drifting quiet fell off a cliff rather than sliding. It is now piecewise-linear on the dBFS axis, so the score tracks the drift continuously.

8.2 Fitting the weights — and what the data actually supports

25/25/20/30 is not derived from data; it is a plausible prior. `training/train_confidence_scorer.py` fits it against speechocean762's human ratings, keeping the sub-score *functions* — which encode real domain knowledge — and fitting only their combination, so the output drops straight back into config.

Measured on 400 utterances, target `fluency` : $R^2 = 0.20$.

Weight	Current → fitted	Verdict
wpm	0.25 → 0.39	Trustworthy — σ 0.32, real variation
pitch_variation	0.25 → 0.38	Trustworthy — σ 0.22
energy	0.20 → 0.10	Unreliable — σ 0.010; studio recordings all at one level
filler_usage	0.30 → 0.13	Unreliable — 95% of clips contain zero fillers

WHY THE WEIGHTS ARE NOT YET ADOPTED

A regression will happily assign a confident weight to a feature that never varies. The script detects near-constant features, refuses to endorse their weights, keeps the existing values, and renormalises the rest. Two of the four numbers here are meaningless, so importing all four would make the score *worse* while looking more scientific.

speechocean762 is read speech scored for pronunciation, not spontaneous public speaking. Closing this properly needs a corpus of rated spontaneous delivery — none is openly available.

8.3 Feedback engine

Threshold rules over five categories — loudness, pitch, pitch variation, speed, fillers, pauses — each producing a message typed `info`, `success`, or `warning`. Overall status derives from the warning count: `excellent` (zero warnings) through `poor` (four or more), with `silent` short-circuiting everything.

Pitch messages are suppressed entirely when nothing voiced was detected, rather than reporting a confident 0 Hz.

8.4 LLM coaching line

`backend/services/llm_feedback.py` — optional, via OpenRouter.

Defended on three fronts, because it sits on the critical path of every chunk:

- **Timeout and single retry** — a slow provider cannot stall the request indefinitely. The original client had no timeout at all.
- **Per-session throttle** — a 2.5 s cadence would otherwise mean ~24 paid calls per minute per speaker. Default minimum interval of 8 s cuts that to ~7, and the slot is reserved under a lock so concurrent chunks cannot both pass.
- **Transcript cap** — prompt size stays bounded regardless of session length.

Every failure path returns `None` and the rule-based message is promoted instead.

9 · Session Layer

`backend/services/session_store.py`

Without sessions every chunk was analysed in isolation and discarded. Four capabilities depend on cross-chunk state:

9.1 Cumulative totals

Filler and keyword counts previously lived only in browser memory and vanished on refresh. They now accumulate server-side and ride back with every response.

9.2 EMA smoothing

A confidence score computed from 2.5 s of audio is statistically noisy — one word is 24 WPM of granularity. Exponential moving averages give a trend line:

```
ema =  $\alpha$  · value + (1 -  $\alpha$ ) · previous      #  $\alpha$  = 0.4, seeded by the first observation
```

The frontend displays `confidence_smoothed` when present. Peak confidence is tracked separately as the raw maximum.

9.3 Adaptive silence gate

A fixed threshold cannot tell a quiet microphone from a quiet room. The session tracks recent chunk loudness and derives a gate from the 10th percentile plus 8 dB headroom:

```
gate = max(HARD_FLOOR, min(ABSOLUTE_SILENCE, p10 + 8))
```

DELIBERATELY ASYMMETRIC

The absolute threshold is a *ceiling*, never a floor. A quiet microphone lowers the gate so speech still reaches the transcriber; a noisy room can never raise it and start transcribing background hiss. The hard floor at -60 dBFS bounds it below.

9.4 Decoder context

The transcript tail feeds the next chunk's `initial_prompt` (§6.3).

9.5 Storage and eviction

In-process, TTL-bounded, guarded by a reentrant lock. Sessions are ephemeral coaching state, not records worth a database; reports are handed to the client as JSON to persist wherever it likes. Expired sessions are dropped on access, and capacity overflow evicts oldest-first.

BUG CAUGHT BY THE TEST SUITE

Pruning ran *before* insertion, so it measured capacity without the incoming session and the store settled one over `max_sessions`. Fixed by inserting first, then pruning.

10 · Deep Learning Layer

`backend/services/ml/` — eight models, all off by default.

10.1 The contract

Every module exposes `is_enabled()` and an analysis function, returns `None` when the feature is off or its dependency is absent, and never raises into the request path. That contract is what keeps a base `pip install -r requirements.txt` fully valid.

10.2 The registry

One component owns loading, caching, and reporting. Lazy (nothing imported until first use — importing torch alone costs seconds), once (per-model load lock), optional (a missing package disables one feature and records why), and serialised (torch modules are not concurrency-safe).

A model that fails to load is never retried, so a broken dependency costs one startup attempt rather than an attempt per request.

10.3 The eight models

Model	Replaces / adds	Warm cost	Notes
CREPE pitch	Autocorrelation pitch	~780 ms	CNN trained to read f0 from waveform; far better on real speech than DSP
Silero VAD	Energy silence gate	~270 ms	Genuinely distinguishes speech from noise
Disfluency	Adds acoustic fillers	—	Requires a trained checkpoint; see §10.4
wav2vec2 SER	Adds emotion, tension	~385 ms	Delivery affect the transcript cannot carry
ECAPA-TDNN	Adds speaker embedding	~375 ms	192-dim; verifies speaker continuity across sessions
pyannote	Adds diarization	session-level	Gated model; needs HF token and accepted terms
openSMILE	Adds 88 eGeMAPS features	~1600 ms	Jitter, shimmer, HNR. Too slow for the live path
WhisperX	Adds word timestamps	varies	Located pauses, articulation rate

YOU CANNOT ENABLE EVERYTHING

The per-chunk models sum to roughly 3.4 s against a 2.5 s budget on CPU. The pairing that earns its cost is **CREPE + Silero (~1050 ms, 42%)**. openSMILE belongs offline, building training features, not on the live path. A GPU changes the arithmetic entirely.

10.4 Why the acoustic filler detector must exist

A STRUCTURAL LIMITATION, NOT A TUNING PROBLEM

Whisper was trained largely on cleaned transcripts and **routinely deletes "um" and "uh"**. Enabling its VAD filter trims the hesitation regions as well. So transcript-based filler counting under-counts *structurally* — no regex improvement can fix it, because the evidence has already been deleted before the text is produced. It survives only in the waveform.

No suitable public checkpoint exists, so `training/train_filler_detector.py` fine-tunes a WavLM backbone into a frame-level classifier on PodcastFillers. Consecutive frames sharing a label collapse into one event, so forty frames of "um" count as one filler.

Merge strategy

Acoustic and text counts merge by **maximum per label, not sum**. When Whisper does keep an "um", both detectors see the same event and adding them would double-count it.

10.5 Two bugs found by running the models

EMOTION LABELS SILENTLY WRONG

`superb/wav2vec2-base-superb-er` emits IEMOCAP abbreviations — `ang`, `hap`, `neu`, `sad` — not full words. The label sets contained only long spellings, so audio classified **98% angry scored tension 0.003**: a furious delivery reported as perfectly calm. Both vocabularies are now mapped, and unrecognised labels surface in `unmapped_labels` with a warning rather than reading as zero.

SPEECHBRAIN BROKEN ON WINDOWS

ECAPA failed with `WinError 1314: A required privilege is not held`. SpeechBrain symlinks model files out of the Hugging Face cache, and Windows refuses without Developer Mode or admin rights. The loader now requests real copies via `LocalStrategy.COPY_SKIP_CACHE`, with a fallback for older releases.

11 · Security & Middleware

backend/middleware.py

11.1 Order

Starlette runs the most recently added middleware first, so registration is inside-out. Execution order: **CORS** → **Request ID** → **Rate limit** → **API key** → **route**. CORS outermost means even a 401 or 429 is properly decorated.

11.2 Protections

Control	Behaviour
Request IDs	Generated or preserved from <code>X-Request-ID</code> , echoed back, bound to a contextvar so log lines trace to the response the caller saw
Rate limiting	Per-client token bucket on <code>/api</code> , default 120/min. Returns 429 with <code>Retry-After</code>
API key	Optional. When <code>API_KEY</code> is set, <code>/api</code> requires <code>X-API-Key</code> , compared in constant time. <code>/api/health</code> stays public so load balancers can probe
Upload caps	Byte limit enforced during streaming; duration limit enforced during decode
Error shape	One JSON shape everywhere; internals never leak

11.3 Vulnerabilities fixed

TRACEBACK DISCLOSURE

The original error handler returned `traceback.format_exc()` in the response body, exposing file paths and internal structure to anyone who could trigger an error. The traceback now goes to logs only; detail appears in responses only under `DEBUG`.

INVALID CORS COMBINATION

`allow_origins=["*"]` with `allow_credentials=True` is forbidden by the CORS specification and rejected by browsers outright. Credentials are now enabled only alongside explicit origins.

TRANSCRIPT HTML INJECTION

The frontend injected transcript text via `innerHTML` for highlighting, without escaping. Text is now escaped before highlight spans are added, and regex metacharacters in filler and keyword names are escaped too.

COST EXPOSURE

`/api/analyze` can spend money at a cloud provider on every call. On a publicly reachable instance, set `API_KEY` — otherwise anyone can drain your Groq and OpenRouter credit.

12 · API Reference

Method	Endpoint	Purpose
POST	<code>/api/analyze</code>	Analyse one chunk. Multipart: <code>audio_file</code> , optional <code>session_id</code>
POST	<code>/api/diarize</code>	Segment a full recording by speaker. Separate from /analyze because 2.5 s cannot separate voices
POST	<code>/api/session</code>	Open a session → <code>session_id</code>
GET	<code>/api/session/{id}</code>	Rolling totals and smoothed averages
GET	<code>/api/session/{id}/report</code>	Full report with stitched transcript
DELETE	<code>/api/session/{id}</code>	Close session, return final report
GET	<code>/api/status</code>	Lightweight readiness probe
GET	<code>/api/health</code>	Deep check: engine, model state, LLM, sessions, ML registry
GET	<code>/docs</code>	Interactive OpenAPI documentation

Audio may be any format libsndfile reads — WAV, FLAC, OGG — at any sample rate.

12.1 Response fields

Field	Type	Meaning
message	string	Headline coaching line (LLM or promoted rule)
pitch , pitch_std	float	Mean and σ over voiced frames, Hz
voiced_ratio	0–1	Fraction of frames carrying pitch
energy , energy_db	float	RMS and dBFS
pause_ratio , longest_pause_sec	float	Silence fraction and longest run
wpm	float	Words per minute for this chunk
fillers	object	Total, rate, per-word breakdown, word count
keywords	object	Total, per-keyword counts, list
clarity , lexical_diversity	int, float	MATTR-based score and raw TTR
confidence_score	0–100	This chunk only
confidence_smoothed	0–100 / null	Session EMA — prefer this for display
confidence_breakdown	object	Four sub-scores, each 0–100
feedback	array	Messages with text , type , category
status	string	silent / poor / needs_improvement / good / excellent
degraded	bool	Part of the pipeline failed; result incomplete
warnings	array	Machine-readable reasons
source_sample_rate	int	Rate of the uploaded audio before resampling
engines	object	Which engine produced each metric
emotion , prosody , disfluency , alignment	object / null	Neural extras; null unless enabled

12.2 Degraded responses

A failed transcription no longer fabricates a confidence score. Acoustic metrics remain genuine, so they are returned and the response is flagged. Warning codes:

Code	Meaning
<code>silence_skipped</code>	Chunk was below the gate; STT deliberately not run
<code>stt_unavailable</code>	Every transcription engine failed
<code>confidence_acoustic_only</code>	WPM and filler sub-scores are not evidence-backed
<code>session_not_found</code>	Supplied session ID expired or does not exist
<code>speaker_changed</code>	Voice no longer matches the session reference embedding

13 · Configuration Reference

`backend/utils/config.py` — `.env` is loaded exactly once, resolved against the project root so behaviour does not depend on the working directory.

BUG THIS FIXED

`load_dotenv` was called twice with different paths. One module resolved the project root; another called it bare, depending on the current directory. Running the server from elsewhere silently disabled the LLM.

13.1 Server & limits

Variable	Default	Purpose
<code>HOST / PORT</code>	0.0.0.0 / 8000	Bind address
<code>DEBUG</code>	false	Include error detail in responses
<code>LOG_LEVEL</code>	INFO	Logger verbosity
<code>CORS_ORIGINS</code>	*	Comma-separated allowed origins
<code>MAX_UPLOAD_BYTES</code>	10485760	Upload size cap
<code>MAX_AUDIO_SECONDS</code>	30	Decoded duration cap
<code>RATE_LIMIT_PER_MINUTE</code>	120	Per-client cap; 0 disables
<code>API_KEY</code>	—	When set, <code>/api</code> requires <code>X-API-Key</code>

13.2 Speech & language

Variable	Default	Purpose
<code>STT_ENGINE</code>	auto	auto / local / groq / openai
<code>GROQ_API_KEY</code>	—	Enables Groq Whisper
<code>OPENAI_API_KEY</code>	—	Enables OpenAI Whisper
<code>WHISPER_MODEL</code>	tiny.en	Local faster-whisper model
<code>WHISPER_COMPUTE_TYPE</code>	int8	Quantisation
<code>WHISPER_VAD_FILTER</code>	true	Trim silence before decoding
<code>OPENROUTER_API_KEY</code>	—	Enables the LLM coaching line
<code>LLM_TIMEOUT_SECONDS</code>	4	Hard timeout
<code>LLM_MIN_INTERVAL_SECONDS</code>	8	Cost cap per session

13.3 Sessions & ML

Variable	Default	Purpose
SESSION_TTL_SECONDS	3600	Idle expiry
MAX_SESSIONS	500	Capacity before oldest-first eviction
SMOOTHING_ALPHA	0.4	EMA factor
HF_TOKEN	—	Gated models and datasets
ML_DEVICE	auto	auto / cpu / cuda / mps
ML_TORCH_THREADS	2	Caps intra-op threads so inference does not starve the request threadpool
ML_WARMUP	true	Load enabled models at startup
ML_PITCH_ENABLED	false	CREPE
ML_VAD_ENABLED	false	Silero
ML_DISFLUENCY_ENABLED / _MODEL	false / —	Acoustic fillers
ML_EMOTION_ENABLED	false	Speech emotion
ML_SPEAKER_ENABLED	false	ECAPA embeddings
ML_DIARIZATION_ENABLED	false	pyannote
ML_PROSODY_ENABLED	false	openSMILE eGeMAPS
ML_ALIGNMENT_ENABLED	false	Word timestamps

14 · Datasets & Training

scripts/datasets.py · DATASETS.md

```
python scripts/datasets.py list      # 14 corpora with size and licence
python scripts/datasets.py check    # what is ready vs. blocked on you
python scripts/datasets.py info podcastfillers
python scripts/datasets.py download speechoccean762
```

14.1 Access classes

Class	Meaning
OPEN	Downloads immediately, no account
HF_AUTH	Free Hugging Face account plus clicking "agree" on the dataset page
REQUEST	A form, academic affiliation, or signature — days, not minutes
PAID	LDC membership or per-corpus fee

WHAT CANNOT BE AUTOMATED

Accepting a licence is a legal act performed by a named person. It cannot be scripted or delegated — the account and the agreement must be yours. Everything *after* acceptance is automated: once `HF_TOKEN` is set, `download` handles the rest.

14.2 The corpora that matter

Dataset	Task	Size	Access	Why
PodcastFillers	Filler detection	~40 GB	Request	The only corpus built for this. Timed um/uh events
speechocean762	Scoring	409 MB	Open ✓	Human delivery ratings, open licence. Downloaded
AMI Meeting	Disfluency	~10 GB	HF auth	Natural disfluent speech, CC BY — commercial-safe
TED-LIUM 3	Public speaking	~54 GB	HF auth	450 h of actual prepared oratory
Common Voice	ASR	~80 GB	HF auth	CC0. Read speech — wrong for filler models
VoxPopuli	Public speaking	~30 GB	HF auth	CC0, formal oratory
LibriSpeech	ASR	~60 GB	Open	No account at all. Read speech, no disfluencies
Switchboard	Disfluency	~25 GB	Paid	Canonical disfluency annotations
Buckeye	Disfluency	~5 GB	Request	Free alternative to Switchboard
RAVDESS / CREMA-D	Emotion	1.5–2 GB	Open	Acted; transfer imperfectly
IEMOCAP / MSP-Podcast	Emotion	12–30 GB	Request	MSP is natural, not acted
VoxCeleb1	Speaker	~35 GB	Request	Rarely needed — use pretrained ECAPA

14.3 Licences drive what you can ship

READ BEFORE TRAINING

A model generally inherits its training data's restrictions. **PodcastFillers and TED-LIUM are non-commercial.** If SpeakTwin is ever commercial, build only on CC0 / CC BY corpora: Common Voice, VoxPopuli, AMI, speechocean762, LibriSpeech.

14.4 Training the filler detector

```
python training/train_filler_detector.py \  
  --data-dir data/podcastfillers --build-manifest \  
  --output-dir models/filler-detector \  
  --backbone microsoft/wavlm-base-plus --epochs 3 --fp16
```

Frame labels are overwhelmingly the negative class, so the script reports positive-class precision, recall, and F1 alongside accuracy. **Accuracy alone will look excellent while the model predicts "none" for everything.**

14.5 Compute

No GPU is needed to *run* SpeakTwin. Training on CPU is not practical.

Option	Notes
Kaggle Notebooks	~30 GPU-hours/week free — best free tier for this size of fine-tune
Google Colab	Free T4; Pro for A100/L4
RunPod / Vast.ai	~\$0.20–0.60/hr; a full fine-tune is a few hours
Modal / Lightning AI	Serverless GPU; same code for training and inference

15 • Frontend

frontend/ — vanilla HTML, CSS, and JavaScript. No framework, no build step.

15.1 Capture chain

```
getUserMedia → AudioContext(16000) → MediaStreamSource
              → ScriptProcessorNode(4096) → accumulate
              → encodeWav(samples, actualSampleRate) → POST
```

The WAV encoder is hand-written: a 44-byte RIFF header followed by 16-bit PCM converted from Float32. Sending WAV rather than the browser's native WebM/Opus means the backend never needs a codec beyond libsndfile.

15.2 Rendering

Element	Behaviour
Energy / pitch meters	20 bars each, height scaled by normalised level
Confidence ring	SVG <code>stroke-dashoffset</code> ; colour steps by band. Prefers <code>confidence_smoothed</code>
Breakdown bars	Four sub-scores as width percentages
Transcript	Escaped, then keyword (green) and filler (red) spans injected
Filler / keyword pills	Server session totals when available; local accumulation otherwise

15.3 Bugs fixed

STATIC ASSETS 404 AT THE ROOT

`index.html` loads `style.css` and `app.js` with relative paths, but the frontend directory was mounted only at `/static` — so from `/` those resolved to `/style.css` and 404'd. The directory is now mounted at both `/static` and `/`, registered after the API router so `/api/*` still wins.

HARDCODED PORT MISMATCH

`API_BASE` pointed at `localhost:8002` while the server defaulted to 8001 and the README said 8000. Now same-origin by default, overridable via `window.SPEAKTWIN_API_BASE`.

16 · Testing & Deployment

16.1 Test suite — 158 tests

File	Covers
<code>test_audio.py</code>	Decode, resample, mono-mix, energy, pitch on known tones, pause detection
<code>test_text_analysis.py</code>	Tokeniser, every filler context rule, keyword n-grams, MATTR stability
<code>test_scoring.py</code>	Confidence curves, sub-score ranges, feedback status transitions
<code>test_session_store.py</code>	Lifecycle, aggregation, smoothing, adaptive gate, eviction
<code>test_api.py</code>	End-to-end: schema, validation, degraded path, sessions, static serving
<code>test_middleware.py</code>	Rate limiting, API key, request IDs
<code>test_ml.py</code>	Optional-by-default contract, registry, merge logic, event collapsing

Tests that need a real neural model are *skipped*, not failed, so the suite stays green on a base install.

16.2 What the tests caught

Three defects were found by the suite rather than in production: the pitch boundary bug (\$5.2), the session eviction off-by-one (\$9.5), and a namespace collision where the registry instance shadowed the registry module, making it unreachable by dotted path.

16.3 Deployment

```
# Base (~1 GB) - DSP + Whisper, no neural extras
docker build -t speakwin .

# With the DL layer (~3-4 GB) - CPU torch + all models
docker build -f Dockerfile.ml -t speakwin:ml .
docker run -p 8000:8000 --env-file .env speakwin:ml
```

The image installs `libsndfile1`, pre-downloads Whisper weights into an image layer, and runs unprivileged as UID 10001 with a health check.

SINGLE WORKER, DELIBERATELY

The session store and rate limiter are in-process. Multiple workers would each keep their own copy — sessions would appear and disappear depending on which worker served the request. Scaling past one replica requires moving both to a shared backend such as Redis.

16.4 CI

Three jobs: the base suite across Python 3.10–3.12 *without* ML extras, an `ml-extras` job that installs CPU torch and asserts the neural layer stays inert by default, and a Docker build.

17 · Technology Choices

17.1 What is used, and why

Choice	Alternatives considered	Reasoning
FastAPI	Flask, Django, Express	Native async, Pydantic validation, automatic OpenAPI
soundfile + scipy	librosa, ffmpeg, pydub	librosa drags in numba (~50 MB) for functions we replaced with 40 lines of numpy. ffmpeg is a system dependency
faster-whisper	openai-whisper, Vosk, wav2vec2	CTranslate2 int8 is ~4× faster than reference Whisper at similar accuracy
Vanilla JS	React, Vue, Svelte	The UI is one page of meters. A build step would cost more than it returns
In-process sessions	Redis, Postgres	Sessions are ephemeral coaching state. A database is the right call only when history must outlive the process
Pydantic schemas	Raw dicts	Untyped dicts are exactly how the README drifted out of sync with its own routes

17.2 What could be used next

Upgrade	Replaces	Value
WebSocket transport	Chunked POST	Chunked POST has a hard ~2.5 s latency floor. Streaming removes it
Redis session store	In-process dict	Unlocks multi-worker and multi-replica scaling
Learned confidence model	Hand-weighted average	speechoccean762 supervision replaces guessed weights (\$8.2)
Word-level timestamps	Frame-based pauses	Locates pauses between specific words; enables playback-synced highlighting
AudioWorklet	ScriptProcessorNode	ScriptProcessorNode is deprecated; AudioWorklet runs off the main thread
Whisper <code>small.en</code> / cloud	<code>tiny.en</code>	Materially better accuracy; costs latency or money
Postgres + auth	No persistence	Progress tracking across sessions and devices
MediaPipe camera module	—	Posture, eye contact, gesture — the placeholder already anticipates it

17.3 Rejected, and why

- **Server-side microphone capture** — impossible on a deployed server; retained only as legacy.
- **librosa in the request path** — heavy dependency for functionality replaced in pure numpy. It returns only as a transitive dependency of torchcrepe in the ML extras.
- **Root TTR for clarity** — length-normalises in the wrong direction for short chunks; MATTR chosen instead.

- **Audio overlap between chunks** — would duplicate words at seams. Decoder prompt carryover achieves the same benefit without the deduplication problem.
- **Summing acoustic and text filler counts** — double-counts the same event; maximum-per-label used instead.

18 · Limitations & Roadmap

18.1 Honest limitations

Limitation	Detail
Confidence weights unfitted	25/25/20/30 is a plausible prior, not a measured result
Filler counts under-report	Until the acoustic detector is trained, Whisper's deletions are invisible
WPM is noisy per chunk	One word is 24 WPM of granularity at 2.5 s. Use the session EMA
English only	<code>tiny.en</code> and every rule assume English
Single worker	In-process session store and rate limiter
No persistence	Sessions die with the process; reports must be saved client-side
Loudness still gain-dependent	dBFS and the adaptive gate reduce but do not eliminate microphone sensitivity
ML budget is tight on CPU	Roughly two models fit inside 2.5 s

18.2 Roadmap

Status	Item	Note
Done	Session history & smoothing	Cross-chunk aggregation with EMA
Done	Export reports	JSON session reports
Done	Configurable thresholds	Every threshold via environment
Done	Deep learning layer	Eight optional models
Next	Train the filler detector	Blocked on PodcastFillers access
Next	Learned confidence scorer	speechocean762 is downloaded and ready
Later	WebSocket transport	Removes the latency floor
Later	Camera module	MediaPipe posture and eye contact
Later	Multimodal fusion	Combined audio + visual score

18.3 Immediate next actions

1. Create a Hugging Face account; accept terms on AMI, TED-LIUM, Common Voice, VoxPopuli; set `HF_TOKEN`.
2. Submit the PodcastFillers request form — the long pole, since it is the only corpus that makes the filler detector trainable.
3. Fit the confidence weights against speechocean762, which is already on disk.
4. Before any public deployment: set `CORS_ORIGINS` to a real origin and set `API_KEY`.

Appendix A · File Map

Path	Responsibility
Application core	
backend/main.py	App construction, CORS, error handlers, health, static mounts, lifespan warmup
backend/middleware.py	Request IDs, rate limiting, API-key auth
backend/schemas.py	Pydantic response models
backend/routes/analyze.py	Analysis endpoint and session lifecycle
Services	
services/audio_io.py	Decode, mono-mix, float32, resample, sanitise
services/audio_analysis.py	Energy, pitch, pause detection
services/speech_to_text.py	Hybrid STT routing and inference locking
services/filler_detection.py	Context-aware filler scanning
services/keyword_detection.py	Target keyword n-gram matching
services/clarity_analysis.py	MATTR clarity scoring
services/confidence_score.py	Weighted composite scoring
services/feedback_engine.py	Threshold-based coaching messages
services/llm_feedback.py	Throttled LLM insight via OpenRouter
services/session_store.py	Session state, smoothing, adaptive gate, reports
services/audio_capture.py	Legacy server-side microphone (unused)
services/camera_placeholder.py	Future MediaPipe body-language module
Deep learning	
services/ml/registry.py	Lazy loading, device resolution, per-model status
services/ml/enrichment.py	Orchestrates every enabled model per chunk
services/ml/pitch.py	CREPE neural pitch
services/ml/vad.py	Silero voice activity detection
services/ml/disfluency.py	Acoustic filler detection and merge logic
services/ml/emotion.py	wav2vec2 speech emotion recognition
services/ml/speaker.py	ECAPA embeddings and pyannote diarization
services/ml/prosody.py	openSMILE eGeMAPS features
services/ml/alignment.py	Word-level timestamps

Utilities, tooling, frontend

<code>utils/config.py</code>	Single source of truth for environment settings
<code>utils/helpers.py</code>	Thresholds, dBFS conversion, logger
<code>utils/text.py</code>	Shared tokeniser with clause awareness
<code>scripts/datasets.py</code>	Dataset manager CLI
<code>scripts/dataset_registry.py</code>	14 corpora: size, licence, access class
<code>training/train_filler_detector.py</code>	WavLM fine-tuning pipeline
<code>frontend/app.js</code>	Web Audio capture, WAV encoding, UI rendering
<code>frontend/index.html</code> · <code>style.css</code>	Layout and glassmorphism dark theme

DOCUMENT PROVENANCE

Every latency figure in this document was measured on the development machine (Windows 11, CPU-only, no NVIDIA GPU) after model warmup, not estimated. Every bug described was found and fixed during development, and each has a regression test. Test count and behaviour reflect the suite at time of writing: **158 passing**.